

SERIALIZABILITY

A schedule is serializable if it is equal to equivalent to a serial schedule. Different forms of schedule equivalence give rise to notion of:

- 1) Conflict serializability
- 2) View serializability

Conflicting Instructions:

- 1) $I_i = \text{read}$ $I_j = \text{read}$ - Non conflicting
 $I_i = \text{read}$ $I_j = \text{write}$ } Conflicting
 $I_i = \text{write}$ $I_j = \text{read}$ }
 $I_i = \text{write}$ $I_j = \text{write}$ }

We say that schedule S is called a conflict serializable if it is conflict equivalent to serial schedule.

Problem 1:

Check whether the schedule is serializable or not

T_1
 $R(A)$

T_2

$R(A)$

T_3

$R(A)$

T_4

$R(A)$

$W(B)$

$W(B)$

$W(B)$

$W(B)$

Checking whether s is conflicting serializable or not

→ Step 1:

$W_1(B), W_2(B)$

$(T_1 \rightarrow T_2)$

$W_1(B), W_3(B)$

$(T_1 \rightarrow T_3)$

$W_1(B), W_4(B)$

$(T_1 \rightarrow T_4)$

$W_2(B), W_3(B)$

$(T_2 \rightarrow T_3)$

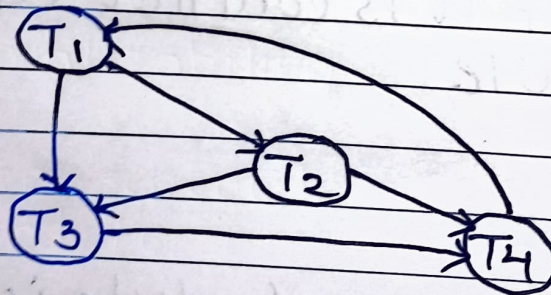
$W_3(B), W_4(B)$

$(T_3 \rightarrow T_4)$

$W_2(B), W_4(B)$

$(T_2 \rightarrow T_4)$

Step 2:



Clearly there exists no cycle in presence so, the given schedule s is connecting schedule.

Hence, it is also a view schedule.

CONCURRENCY PROBLEMS IN DBMS

Concurrency problems are

Concurrency problems in Transaction.

- Dirty Read Problem
- Unrepeatable Read Problem
- Least Update Problem
- Phantom Problem

* Least Update Problem

A write without performing a read operation is called blind write.

serializability scheduler behave exactly as same as serial scheduler

Thus, they are always:

- 1) Consistent
- 2) Reliable
- 3) Cascadess
- 4) Strict

Check whether a schedule is conflict serializable or not:

- 1) Find all list of conflicting operation
- 2) Start creating a precedence graph by drawing one node for each transaction.
- 3) Draw an edge for each conflict pair such that $X_i(V)$ & $Y_j(V)$ forms a conflict pair then draw an edge from T_i to T_j . Thus ensure that T_i gets executed before T_j .
- 4) Check if there is only one cycle formed in the graph. If there is no cycle formed, the graph is conflict serializable else not.

View Serializability

If a given schedule is found to be view equivalent to some serial schedule, then it is called as view serializable schedule.

View Equivalent Schedules

Condition 1:

For each data item X , if transaction T_i reads X from the database initially in schedule S_1 , then in schedule S_2 also, T_i must perform the initial read of X from the database.

Condition 2:

If transaction T_i reads a data item that has been updated by the transaction T_j in schedule S_1 , then in schedule S_2 also, transaction T_i must read the same data item that has been updated by the transaction T_j .

Condition 3:

For each data item X , if X has been updated at last by transaction T_i in schedule S_1 , then in schedule S_2 also, X must be updated at last by transaction T_i .

Checking Whether a Schedule is View Serializable or Not:

Method 1:

- ① Check whether the given schedule is conflict serializable or not.
- ② If the given schedule is conflict serializable, then it is surely view serializable.
- ③ If the given schedule is not conflict serializable, then it may or may not be view serializable. Go and check using other methods.

Method 2:

- ① Check if there exists any blind write operation.
- ② If there does not exist any blind write, then the schedule is surely not view serializable.
- ③ If there exists any blind write, then the schedule may or may not be view serializable.

Method 3:

- ① In this method, try finding a view equivalent serial schedule.
- ② Using above 3 conditions, write all dependencies.
- ③ Draw a graph using dependencies.
- ④ If there exists no cycle in graph, the schedule is view serializable else not.

Problem - 02.

Check whether the given schedule S is view serializable or not.

T_1
 $R(A)$

T_2
 $R(A)$

T_3
 $W(A)$

$W(A)$

→ Step 1: (Method 1)

$R_1(A), W_3(A)$

$R_2(A), W_3(A)$

$W_3(A), W_1(A)$

$R_2(A), W_1(A)$

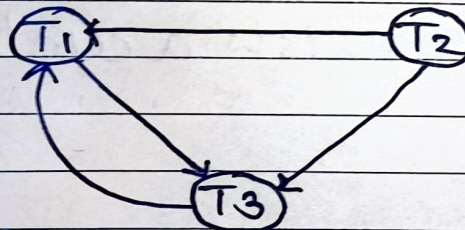
$(T_1 \rightarrow T_3)$

$(T_2 \rightarrow T_3)$

$(T_3 \rightarrow T_1)$

$(T_2 \rightarrow T_1)$

Step 2:



As cycle exists, it is not conflict serializable.

Method 2:

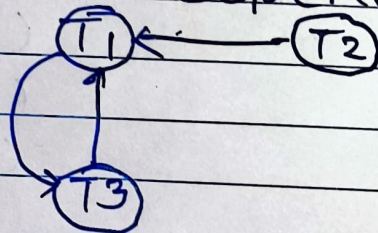
There is a blind write between $W_3(A)$ & $W_1(A)$.

Since, $W_1(A)$ & $W_3(A)$ are conflict pairs, Method 2 is invalid.

Method 3:

Drawing a Dependency Graph:

- ① T1 firstly reads A and T3 firstly updates A. So, T1 must execute before T3
- ② Thus, we get the dependency $T1 \rightarrow T3$
- ③ Finally, updating on A is made by the transaction T1. So, T1 must execute after all other transactions
- ④ Thus, we get the dependency $(T2, T3) \rightarrow T1$. There exists no write-read sequences
- ⑤ Drawing the dependency graph:



Clearly, there exists a cycle in the dependency